Department of Artificial Intelligence and Machine Learning

Acharya Institute of Technology

Soladevanahalli, Bengaluru 560 107

Project Phase-II : Interim Presentation

on [Presentation Date]

ACHARYA

Dept of AI &ML

TraceWeave-Core: Measuring What Static Call-Graph Extraction Misses in Real Python Repositories

Presented by

Group No	Name	USN
[Group No]	[Member 1 Name]	[1AY__AI___]
	[Member 2 Name]	[1AY__AI___]
	[Member 3 Name]	[1AY__AI___]
	[Member 4 Name]	[1AY__AI___]
Guide Name: **Designation:**		

ACHARYA

Dept of AI & ML

Contents

1 **Problem Statement & Objective**
Why static call graphs are the bottleneck

2 **Flow / Methodology**
The measurement pipeline, and what makes it a measurement

3 **Demo of Project**
A call graph missing a real call, live

4 **Results / Output**
Phase 0, measured

5 **Status of Paper**
Submission and proof

6 **Proposed vs Actual Outcome**
Targets, deliveries and the surprises

7 **Conclusion, Deliverables & Impact**
What ships and who it helps

8 **Individual Responsibilities**
Workstream ownership

9 **Gantt Chart**
Workstreams delivered, and what remains

10 **References**
IEEE format

Content is presented as bullets, figures and tables throughout — no paragraphs.

ACHARYA

Dept of AI & ML

1. Problem Statement & Objective

THE PROBLEM

- Repository-level modernization is bottlenecked by call-graph quality, not model capacity.
- Python decides many calls at run time — getattr dispatch, importlib, plugin registries, DI containers.
- Static extractors cannot see those edges — and no one has measured how many are lost.
- A silently lossy graph makes an LLM code agent confidently wrong.

WHAT ONE MISS LOOKS LIKE

STATIC (PyCG 0.0.8)

```
main → dispatch
dispatch → <builtin>.getattr
```

RUNTIME (sys.setprofile)

```
main → dispatch
dispatch → handle_email
```

✕ **the second edge is real, executed, and invisible to static analysis**

OBJECTIVES

O1

Measure the miss rate

Recall of static extraction against runtime-observed edges (H2a)

O2

Name the mechanism

Show misses concentrate in identifiable construct classes

O3

Release DynBench-Core

Pinned repos, replayable workloads, reference edge sets, harness

O4

Report feasibility

What it costs to trace real projects at corpus scale

ACHARYA

Dept of AI & ML

2. Flow / Methodology — Measurement Pipeline (1 of 2)

1

Discover

GitHub search over Python repos with a build file.

→ **558 candidates queued**

2

Score density

AST scorer counts dynamic constructs per KLOC of source only.

→ **LOW / MED / HIGH stratum**

3

Triage

Containerised install + test run, one repo, one Python, one attempt.

→ **GREEN / AMBER / RED + pinned lockfile**

4

Capture workloads

Annotators record ≥3 replayable runs per repo, verified twice.

→ **The execution oracle**

5

Extract both graphs

Static: PyCG 0.0.8 + Pyright 1.1.411. Dynamic: sys.monitoring tracer.

→ **S and R**

6

Fuse & measure

Dual-state CPG, fused on read; recall over the comparable universe.

→ **The H2a number**

Every stage runs inside a container pinned by digest, never by tag — **the host never traces.**

ACHARYA

Dept of AI & ML

2. Flow / Methodology — What Makes It a Measurement (2 of 2)

IDENTITY IS PINNED BEFORE ANYTHING RUNS

Repo identity — all five required

```
(repo_url, sha, pyver,
 base_image_digest, lockfile_sha256)
```

Node identity — frozen

```
(repo_url, sha, relative_path,
 qualified_symbol)
```

Line numbers are deliberately excluded — they move under reformatting and differ between the static parse and the runtime frame.

Both graphs are extracted inside the same pinned image, so a difference between them is a property of the code — not of the environment it ran in.

METRIC DISCIPLINE, PRE-REGISTERED

- **Comparable universe —** both ends in-repo and Python-level. C/builtin and third-party are excluded from the primary denominator.
- **Recall = |S∩R| / |R|.** |S∩R|/|S| is never called precision — it is the workload-confirmed fraction.
- **R is a lower bound,** conditional on the recorded workloads. That wording is mandatory everywhere.
- **Wrapper transparency —** caller → wrapper → f collapses to caller → f, so decorators cannot manufacture misses.
- **No LLM-as-judge** anywhere in validation or headline metrics.

FROZEN BEFORE THE CODE THAT CONSUMES THEM

`metric_spec_h2a.md`
frozen **v1.0**

Node identity, what counts as an edge, the comparable universe, threats pre-empted.

`density_scorer_spec.md`
frozen **v1.0**

Thresholds calibrated once, on 14 reference repos, before any corpus admission.

`config/base_images.json`
frozen **v1.2**

Five image digests, tool pins, the recipe ladder. Appended to, never edited.

3. Demo of Project — A Call Graph Missing a Real Call

ACHARYA

Dept of AI & ML

WHAT RUNS ON SCREEN

1. **fixture.py**
   A small plugin registry that dispatches through getattr — the construct class H2a is about.

2. **PyCG 0.0.8**
   Runs in the pinned container against the same file, writes static.json.

3. **sys.setprofile tracer**
   Executes the fixture for real, writes runtime.json.

4. **Web explainer**
   React + Cytoscape diffs the two graphs and highlights the edge only one of them has.

REAL OUTPUT, THIS REPOSITORY

`static.json` — 3 edges

```
fixture → fixture.main
fixture.main → PluginRegistry.dispatch
PluginRegistry.dispatch → <builtin>.getattr
```

`runtime.json` — 3 edges

```
fixture → fixture.main
fixture.main → PluginRegistry.dispatch
PluginRegistry.dispatch → PluginRegistry.handle_email
```

✕ **handle_email is executed every run and appears in no static graph.**

DEMO SAFETY

Warm end-to-end run 0.35–2.7 s · 15 s fallback threshold · `restore_fallback.sh` replays a recorded pair.

The demo is the hypothesis in miniature. Run the same static-vs-runtime diff across the whole pinned corpus and the result is the H2a number.

ACHARYA

Dept of AI & ML

4. Results / Output — Phase 0, Measured

558
candidates discovered & scored
target ~450 · floor 300

100%
PyCG failure rate under Python 3.12
it does not run there at all

1.6%
PyCG failure on library code under Python 3.10
3.1% on test & fixture paths

93 s
click triaged GREEN, end to end
9-package lockfile + sha256

STRATIFIED QUEUE & SCORER CALIBRATION

Stratum	Queue	Ref. set
HIGH (≥10.0)	135	6
MED (4.0–10.0)	177	4
LOW (<4.0)	246	3
TOO_SMALL (<500 SLOC)	—	1
Total	**558**	**14**

Stratum comes from source only — scoring test suites together made every pytest project look HIGH.

FOUND BY RUNNING PHASE 0, NOT PLANNING IT

- **PyCG 0.0.8 cannot be imported on Linux —** it ships PyCG/ but imports pycg. A symlink shim is now mandatory in every image.
- **The scorer's first run had no discrimination —** 10 of 14 repos landed HIGH. Four false-positive classes were fixed before freezing.
- **It corrected two a-priori labels, and was right:** requests really does call importlib.import_module in its compat layer.
- **ins1's classifier has an ordering bug.** Kept verbatim as error_class_v10 for audit comparability; the disagreement rate is now reportable.
- **A 16.8× overcount was caught in verification —** click's collected tests read as 32965 instead of 1965.

ACHARYA

Dept of AI & ML

5. Status of Paper (1 of 2)

PAPER 1 · SYSTEMATIC REVIEW

Graph-Aware Neuro-Symbolic Approaches for Legacy Modernization: A Systematic Review

Adarsha S P · Sankalp Shankar · Surabhi Singh · Pravith S Upadhyaya · Abhishek Kumar

Scope: IEEE Xplore, ACM DL and arXiv, January 2020 – April 2026; repository-level code translation, refactoring and validation. §VIII.B is the finding this project turns into a measurement.

STATUS

UNDER REVIEW

Venue: [Venue Name]
Submitted: [Date]
Paper ID: [ID]

PAPER 2 · IN PREPARATION

TraceWeave-Core: How Much of a Python Call Graph Does Static Extraction Miss?

Gated on the Phase 3 GO / NO-GO. The headline is the H2a recall figure with its per-stratum and per-edge-type breakdowns, plus the DynBench-Core artifact. The measurement is publishable either way — a low miss rate concentrated at named constructs is a stronger result than a diffuse high one.

No threshold or definition may be tuned after seeing results — the specs were frozen before the extraction code existed.

5. Status of Paper — Proof (2 of 2)

ACHARYA

Dept of AI & ML

SUBMISSION PROOF

[Paste the submission-confirmation screenshot here]

Acknowledgement e-mail or the portal's submitted-papers page

WHAT THE PROOF MUST SHOW

- ☐ Paper title exactly as submitted
- ☐ Full author list in submission order
- ☐ Venue name and track
- ☐ Submission or paper ID
- ☐ Date-stamped acknowledgement
- ☐ Current state shown by the portal

Redact reviewer identities and any e-mail address that is not a team member's before the slide is shown.

6. Proposed Outcome vs Actual Outcome (1 of 2)

PHASE 0 COMMITMENT	PROPOSED	ACTUAL	
Candidates discovered & scored	~450 (floor 300)	558 queued	MET
Stratum balance — HIGH not thin	risk flagged in plan	HIGH 135 / MED 177 / LOW 246	MET
Pyright pinned before Phase 1	version query pending	1.1.411, published 2026-06-25	MET
PyCG parse-failure rate measured	"expected non-trivial"	1.6% @ 3.10 · 100% @ 3.12	MET
triage/runner.py — parallel queue	recipe ladder + checkpoints	delivered, re-verifies every outcome	MET
Density scorer calibrated & frozen	calibrate once, then freeze	v1.0 — 6 HIGH / 4 MED / 3 LOW / 1 TS	MET
Corpus with ≥3 workloads per repo	~60 repos (floor 40)	Phase 1 work — not started	NEXT
H2a recall figure	Phase 3 GO / NO-GO	not yet measured, by design	NEXT

Phase 0 closed on schedule and over target on every item it owned. The two NEXT rows are Phase 1 and Phase 3 deliverables, not slippage.

ACHARYA

Dept of AI & ML

ACHARYA

Dept of AI & ML

6. What the Plan Did Not Predict (2 of 2)

1 PyCG does not run on Python 3.12 at all

1311 of 1311 reference files fail with ImportManagerError — not a SyntaxError, so §2 anticipated the wrong risk.

→ Static baselines now run on ≤3.11 while repos are triaged on 3.12. Recorded as deliberate, not discovered from a stack trace.

2 The published 0.0.8 sdist is unimportable on Linux

It installs PyCG/ but every internal import reads pycg. It only ever worked on case-insensitive macOS.

→ Pin held at 0.0.8; every image carries the symlink shim, and the deviation is written into MANIFEST §2.1.

3 This Mac cannot build the corpus

8 GB RAM OOM-kills test suites at --jobs 3; the corpus must be built on one architecture.

→ Mac is the manager host; the Linux x86_64 box executes. The corpus is amd64 and a Mac-triaged repo is development output.

4 A patch introduced a silent 16.8× overcount

n_collected matched pytest's pre-deselection total: click read 32965 instead of 1965.

→ Caught during verification, fixed in both triage.sh and runner.py; the total is kept separately as n_collected_total.

STILL OPEN — CARRIED INTO PHASE 1

- **Amend ins1 §6.** triage.sh v1.2 emits OOM, TIMEOUT, KILLED and collect_partial, which the operator sheet forbids — an operator cannot record what the script prints. validate_sheet.py --v12 is the interim machine check.
- **Re-run the 10% human-vs-classifier audit under v1.2,** and do not pool the 14 v1.1 spot-check rows with the 558.

ACHARYA

Dept of AI & ML

7. Conclusion, Deliverables & Impact

Key findings

- Phase 0 closed complete and over target: 558 candidates against a plan of ~450.
- The field's standard static baseline fails on 100% of files under the interpreter this project defaults to.
- Dynamic-feature density is measurable and discriminating once four false-positive classes are removed.
- Every headline defect so far was found by running the pipeline, not by reviewing it.

Deliverables

- DynBench-Core — pinned repos, replayable workloads, reference edge sets, harness.
- A pre-registered H2a metric specification, frozen before any extraction code.
- Automated triage: triage.sh v1.2 + runner.py with explicit corpus admission.
- A tracing-feasibility survey, including the repos that could not be traced.

Impact

- Tells graph-conditioned code agents how lossy their retrieval substrate really is.
- Names the construct classes to fix, rather than reporting one diffuse percentage.
- Benchmarks outlive methods — the artifact stays useful after the number is quoted.
- A negative result is still the paper: the conditional claim is the strong one.

Every pin, digest and frozen threshold exists so the published number is a measurement — **not the result of searching until something looked good.**

8. Individual Responsibilities

NAME	USN	WORKSTREAM	OWNS
[Member 1 Name]	[1AY__AI___]	**Discovery & stratification**	discover_candidates.py · density_score.py · scorer calibration and the frozen thresholds
[Member 2 Name]	[1AY__AI___]	**Triage automation**	triage.sh v1.2 · runner.py · lockfile pinning, corpus admission and the 10% audit
[Member 3 Name]	[1AY__AI___]	**Static baselines**	PyCG 0.0.8 + Pyright 1.1.411 in pinned images · parse-rate measurement · feasibility survey
[Member 4 Name]	[1AY__AI___]	**Tracing, demo & specification**	sys.monitoring tracer · live demo and web explainer · metric_spec_h2a.md and MANIFEST

Guide — [Guide Name], [Designation]. Reviews every freeze and version bump, approves the Phase 3 GO / NO-GO, and supervises submission.

Shared across all four: weekly integration, the 10% human-vs-classifier audit, and rehearsal of the live demo on the presentation machine.

ACHARYA

Dept of AI & ML

9. Gantt Chart — Workstreams to the Week 8 Seam



Eight of twelve workstreams are delivered, in the first three weeks. All four of MANIFEST §7's Phase-1 blockers — the Pyright pin, the PyCG parse-rate measurement, runner.py and discover_candidates.py — were cleared before Phase 1 opened.

The chart stops at the Week 8 seam because H2a, DynBench-Core and the feasibility survey already form a complete paper there. H2b-lite and the release run weeks 8–12 beyond it.

ACHARYA
Dept of AI & ML

ACHARYA

Dept of AI & ML

10. References

[1] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical call graph generation in Python," in Proc. 43rd Int. Conf. Software Engineering (ICSE), 2021, pp. 1646–1657.

[2] A. S. P, S. Shankar, S. Singh, P. S. Upadhyaya, and A. Kumar, "Graph-aware neuro-symbolic approaches for legacy modernization: A systematic review," 2026, under review.

[3] M. Shannon, "PEP 669 — Low impact monitoring for CPython," Python Enhancement Proposals, Python Software Foundation, 2022.

[4] R. Bairi, A. Sonwane, A. Kanade, S. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, "CodePlan: Repository-level coding using LLMs and planning," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[5] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, "RepoGraph: Enhancing AI software engineering with repository-level code graph," in Proc. Int. Conf. Learning Representations (ICLR), 2025.

[6] Y. Pan, J. Zhao, C. Wang, and Q. Liu, "RepoTransBench: A real-world benchmark for repository-level code translation," arXiv:2412.17744, 2024.

[7] A. R. Ibrahimzada, K. Ke, M. Pawagi, M. S. Abid, R. Pan, S. Sinha, and R. Jabbarvand, "Repository-level compositional code translation and validation," arXiv:2410.24117, 2024.

[8] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representations with data flow," in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[9] Microsoft, "Pyright: Static type checker for Python," version 1.1.411, 2026. [Online]. Available: https://github.com/microsoft/pyright

[10] Python Software Foundation, "ast — Abstract syntax trees," Python 3.12 documentation, 2026. [Online]. Available: https://docs.python.org/3.12/library/ast.html

IEEE reference format. Internal project documents (MANIFEST.md, metric_spec_h2a.md, density_scorer_spec.md) are cited in the report, not here.

ACHARYA

Dept of AI & ML

Thank you